

TREES AND THEIR DEGREES

MARIE-CAMILLE DELARUE

CONTENTS

1. Introduction	1
2. Trees	2
2.1. Trees with simple junctions	2
2.2. Trees with double junctions	2
2.3. Goal – notion of degree	3
3. Defining the detector map	5
4. Degree of a map between spheres	5
4.1. Degree formula	5
4.2. Algorithm	7
4.3. Picking a regular value	7

1. INTRODUCTION

In the course of our study of the family of Higman–Thompson groups, there came to be a question that was very easy to answer in small dimensions and much harder in higher dimensions. The fundamental reason for this was that it was easy to come up with a reasonable visualization in low dimensions, and much harder in higher dimensions because of the limitations of 3-dimensional space. Therefore, we tried to come up with a good visualization in higher dimensions.

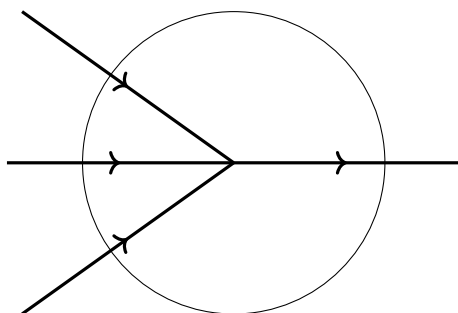
The following document explains this problem and the framework we came up with to answer it in higher dimensions. For extra context on the Thompson groups and their homology, please consult the other file in this folder. Reading the following document is however completely necessary to understanding the specific question we were working on as well as its implementation.

2. TREES

We will consider finite rooted trees, and we call an **a -junction** a vertex of the

tree with one parent and a children: For instance,  is a 3-junction.

2.1. Trees with simple junctions. Let us start by considering the following problem. Consider a finite rooted tree which only has a -junctions, for a an integer. We will think of trees as being oriented from the leaves to the root. Consider the following picture:



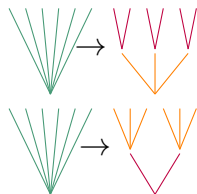
We will call the **degree** of a junction the number of incoming edges minus the number of outgoing edges in a small circle around the junction. For any a -junction, this number is clearly $a - 1$.

Now let us assume we have trees with a -junctions and b -junctions. The a -junctions have degree $a - 1$, and the b -junctions have degree $b - 1$.

2.2. Trees with double junctions. Now let us consider a family of trees with both a -junctions and b -junctions, for a and b two different integers, with the following additional **interchange relation**:

$$\begin{array}{c} \text{V} \quad \text{V} \quad \text{V} \\ \diagdown \quad \diagup \quad \diagdown \\ \text{V} \end{array} = \begin{array}{c} \text{V} \quad \text{V} \quad \text{V} \\ \diagdown \quad \diagup \quad \diagup \\ \text{V} \end{array}.$$

How can we represent this equality topologically? In topology, we can always relax an equality between two objects by creating a path between them. We add an ab -junction which represents this path:



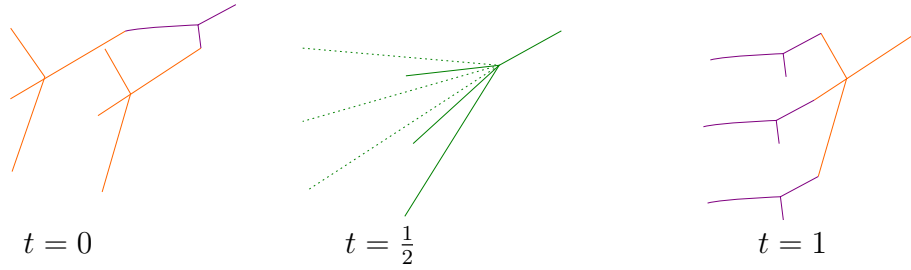


FIGURE 1. The tree is parametrized by $(x, t) \in [0, 1]^N \times [0, 1]$. The barycenter of the junctions is governed by x and the shape of the tree (including how far apart the junctions are) is governed by t .

The idea is that if our tree lives in dimension N , we are adding an extra dimension which represents time. The position of t in $[0, 1]$ indicates what the form of the tree is in a continuous way as depicted in figure 1. Now our tree is parametrized by an element in $[0, 1]^{N+1}$.

In the program, there is a function `make_family` which holds the combinatorial information about the tree, and a function `tree_at` which to a point in $[0, 1]^{N+1}$ and the combinatorial information of the tree associates the corresponding embedding of a tree inside a unit cube $[0, 1]^N$.

2.3. Goal – notion of degree. We would like to determine the degree of this extra junction. It is tempting to say that the degree of this junction is $ab - 1$. However, this does not really work, in part because our definition of degree is not precise enough. Let's assume we have an ab -junction. In a neighborhood of this ab -junction however, we have two very different pictures. So if we move just a little bit away from the ab -junction, we can end up in two very different scenarios which have different degrees.

Let's give another definition of the degree by going back to the case of a simple junction. If we imagine a smaller lens traveling around the junction and recording what it sees, we will see a single path $a + 1$ times: a time with positive orientation and once with negative orientation, and so we recover a degree of $a - 1$.

We will find the degree of the ab -junction by traveling around it and recording what we see. Because the ab -junction lives in "higher dimension", we need to count the number of a -junctions and b -junctions that we see around it, not just paths.

Explain this better. Also I should try to make a picture for the bigger junctions but I'm just not sure how, the point is that it's hard

This is the basis of the heuristic we will follow to define the detector map.

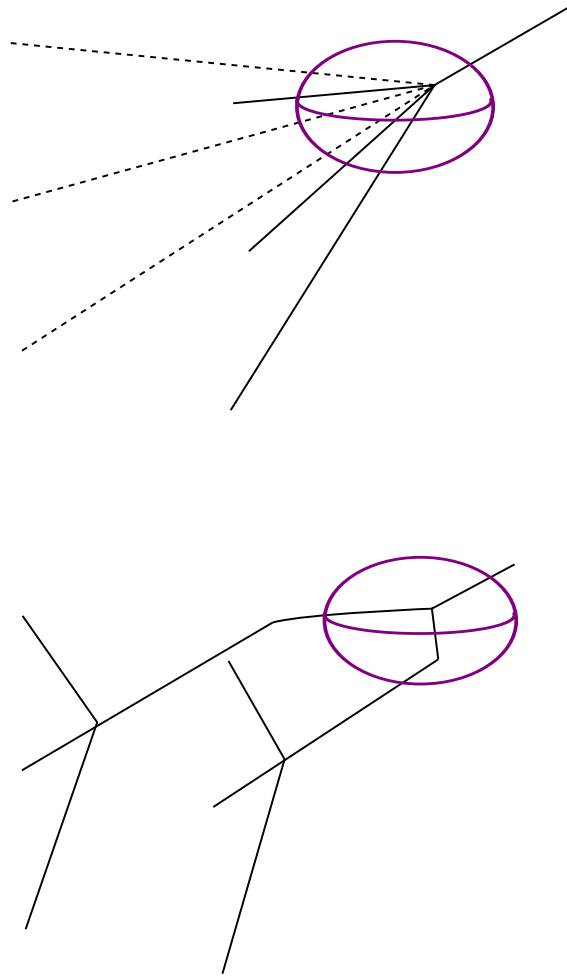


FIGURE 2. Two trees very close to each other but with very different "local images". The trees are oriented from leaves to root.

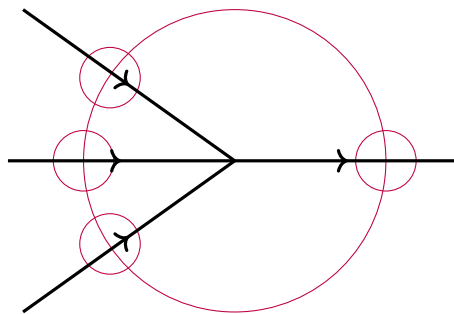


FIGURE 3. The case of a simple junction

3. DEFINING THE DETECTOR MAP

We create a detector for a -junctions, and one for b -junctions, according to the following procedure. Given a specific embedding of a tree, we look at a smaller detector cube centered inside the unit cube and we record what it contains:

- if it contains a single a -junction and no other connected component of the tree, we record the position of the a -junction, which is a point inside the detector cube, and rescale it to a point of the unit cube.
- if it contains anything else, we record "nothing", which we set as the basepoint.

Therefore, the range of the detector map is a unit cube $[0, 1]^N$, whose boundary has been collapsed to a point, which we call the basepoint.

The detector map that we define in the program is actually a composition of `tree_at` and the aforementioned procedure:

- Take a point in the cube $[0, 1]^{N+1}$ that we can write (x, t) where $x \in [0, 1]^N, t \in [0, 1]$;
- Construct an embedding of a tree inside the cube $[0, 1]^N$. The barycenter of the junctions is at x and the shape of the tree is determined by t ;
- Look inside a smaller detector cube and record what you see;
- Get a point in $([0, 1]^N)/(\partial[0, 1]^N)$.

This is what the functions `detect_a` and `detect_b` correspond to.

4. DEGREE OF A MAP BETWEEN SPHERES

The notion of degree we defined in section 2 is closely related to the notion of degree of maps between spheres. Basically, the degree of a map from a sphere S^N to itself is the number of times the map goes around the target sphere, up to homotopy. It is clearer to see this on an example.

Example 4.1. Imagine the function from the circle S^1 to another S^1 which wraps the circle around itself twice, as illustrated in figure 4: The degree of this map is 2.

Examples on spheres of higher dimension are more difficult to understand than in dimension 1, but they work in the same way.

4.1. Degree formula. If the map is C^1 , we can obtain the degree by picking a regular point y in the image (i.e. a point with nonsingular differential), and counting its preimages:

$$d = \sum_{p \in f^{-1}(y)} s_p$$

where s_p is the sign of the Jacobian of f at point p .

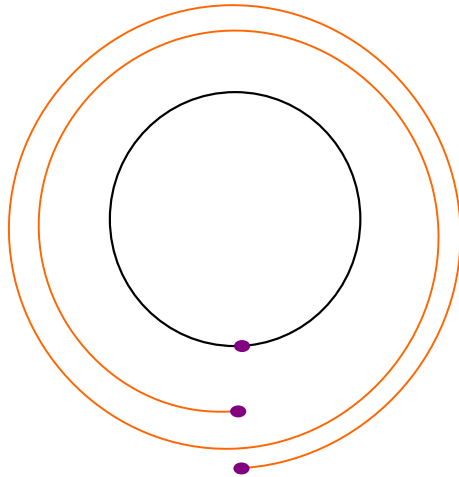


FIGURE 4. The orange circle wraps twice around the first one, starting and ending at the purple point.

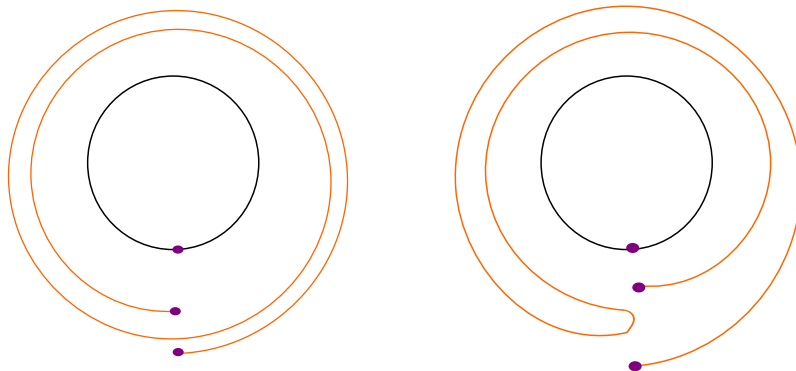


FIGURE 5. The left map is the one introduced earlier. The map on the right wraps the orange circle around the black circle once, then changes directions and wraps around it once more.

Example 4.2. Consider the two maps from S^1 to S^1 depicted in figure 5: If one were simply to count the preimages at any regular value – for instance, not at the basepoint of the circle (the purple point), one would see that for both maps, there are two points of the orange circle mapped to the same point of the black circle. However, the degree of the left map is 2, and the degree of the right map is 0. This is because in the first instance, both preimages contribute the same sign to the formula: they are oriented in the same way. In the second instance, they have opposite orientation.

Recall the detector map that goes from $[0, 1]^{N+1}$ to $[0, 1]^N / \partial[0, 1]^N$. If we restrict to the boundary of $[0, 1]^{N+1}$, we get a map from something which is equivalent to an N -sphere to another space which is equivalent to an N -sphere. This is why the notion of degree of maps between spheres applies in this case.

4.2. Algorithm. Let us now go through the idea of the algorithm. Given a regular value y , the algorithm looks for the preimages of y in the source. To do this, it cuts up the source into many small little cubes and looks for the preimage inside each cube. Of course, searching for the preimage inside every cube would take too much time, so many of them are discarded.

4.3. Picking a regular value. It is very important to pick a good regular value to carry out the above procedure.